# Technical Documentation: Workshop 5

## Building an Offline-First Queue App with Geolocation in Flutter

**Project**: `waiting_room_app` (TP5)

**Author**: Iheb Najjar

**Class**: DSI 31

**Framework**: Flutter 3.44.8 (Dart SDK ^3.12.2)

**Target Platforms**: Web (Edge/Chrome), Android, iOS, Windows

---

## 1. Project Overview & Architecture

Workshop 5 extends the real-time waiting room application developed in Workshop 4 into an **Offline-First**, location-aware queue management system.

```
flowchart TD
    UI["WaitingRoomScreen (Flutter UI)"] -->|Actions| QP["QueueProvider (ChangeNotifier)"]
    QP -->|Immediate Local Insert| LQS["LocalQueueService (SQLite / sqflite)"]
    QP -->|GPS Capture| GS["GeolocationService (geolocator)"]
    QP -->|Non-blocking Async Upsert| SB["Supabase Remote Database"]
    SB -->|Realtime Postgres Changes| QP
    QP -->|notifyListeners()| UI
```

### Key Architectural Highlights

1. **Offline-First Storage**: New clients are persisted locally in SQLite (`sqflite`) before remote operations occur. The app functions seamlessly even when offline.
2. **Non-blocking Background Sync**: Once local storage succeeds, a non-blocking asynchronous routine (`unawaited(_syncLocalToRemote())`) synchronizes unsynced records to Supabase using `upsert`.
3. **GPS Location Capture**: Captures latitude and longitude with 4-decimal precision (approx. 11-meter accuracy). If location services are disabled or permissions denied, the app degrades gracefully without crashing.
4. **Optimistic UI Updates**: State mutations reflect instantly in the user interface to ensure zero latency for end-users.
5. **Cross-Platform Compatibility**: Fully verified on Web (Microsoft Edge) with transparent memory fallback, Windows Desktop via SQLite FFI, and Mobile devices (Android/iOS).

---

## 2. Database Schemas

### 2.1 Local SQLite Schema (`waiting_room.db`)

Table: `local_clients`

```
CREATE TABLE local_clients (
    id TEXT PRIMARY KEY,
    name TEXT NOT NULL,
    lat REAL,
    lng REAL,
    created_at TEXT NOT NULL,
    is_synced INTEGER NOT NULL DEFAULT 0
);
```

- `id`: Standard UUID v4 string.
- `name`: Client full name.
- `lat`, `lng`: Real numbers representing GPS coordinates.
- `created_at`: ISO 8601 formatted timestamp string.
- `is_synced`: Flag (`0` = pending sync, `1` = synced with remote Supabase).

### 2.2 Remote Supabase Schema (`clients`)

```
CREATE TABLE public.clients (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    lat DOUBLE PRECISION,
    lng DOUBLE PRECISION,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL
);

-- Enable Row Level Security (RLS)
ALTER TABLE public.clients ENABLE ROW LEVEL SECURITY;

-- Allow anonymous and authenticated read/write access
CREATE POLICY "Allow public read" ON public.clients FOR SELECT USING (true);
CREATE POLICY "Allow public insert" ON public.clients FOR INSERT WITH CHECK (true);
CREATE POLICY "Allow public update" ON public.clients FOR UPDATE USING (true);
CREATE POLICY "Allow public delete" ON public.clients FOR DELETE USING (true);
```

---

## 3. Class & Component Catalog

### 3.1 `Client` (`lib/models/client.dart`)

Encapsulates queue item data:

- `id`: Unique identifier (UUID).
- `name`: Client name.
- `lat`, `lng`: Geolocation coordinates (optional).
- `createdAt`: Creation timestamp.
- `isSynced`: Local synchronization indicator.
- `toMap()`: Serializes model for SQLite and Supabase payload.
- `fromMap()`: Deserializes database records, handling numeric type conversions safely.

### 3.2 `LocalQueueService` (`lib/local_queue_service.dart`)

Manages SQLite persistence:

- `database`: Asynchronously retrieves or initializes the database instance.
- `insertClientLocally(Map<String, dynamic>)`: Inserts or replaces a record with `ConflictAlgorithm.replace`.
- `getClients()`: Returns all stored clients ordered chronologically (`created_at ASC`).
- `getUnsyncedClients()`: Returns clients where `is_synced = 0`.
- `markClientAsSynced(String id)`: Updates `is_synced = 1`.
- `removeClientLocally(String id)`: Deletes record from SQLite.
- `clear()`: Wipes the table (used for test teardowns).
- `_webFallback`: Handles web execution (Edge/Chrome) seamlessly when native SQLite C-libraries are unavailable.

### 3.3 `GeolocationService` (`lib/geolocation_service.dart`)

Handles location detection:

- `getCurrentPosition()`:

6. Requests `checkPermission()` and `requestPermission()`.
7. Verifies `isLocationServiceEnabled()`.
8. Queries position using `LocationSettings(accuracy: LocationAccuracy.low, timeLimit: Duration(seconds: 5))`.
9. Safely catches platform exceptions, timeouts, or uninitialized bindings and returns `null` gracefully.

### 3.4 `QueueProvider` (`lib/queue_provider.dart`)

State manager extending `ChangeNotifier`:

- `initialization`: Future tracking local database initialization.
- `_loadQueue()`: Loads clients from SQLite, triggers remote sync, and establishes real-time postgres listener.
- `addClient(String name)`: Optimistically adds client, acquires GPS position, saves to SQLite, notifies listeners, and schedules background sync.
- `removeClient(String id)`: Removes locally, notifies UI, and triggers remote deletion on Supabase.
- `nextClient()`: Dequeues the first waiting client.
- `_syncLocalToRemote()`: Queries unsynced records from SQLite and performs `upsert` against Supabase.

### 3.5 `WaitingRoomScreen` (`lib/main.dart`)

The user interface:

- Input field and "Add" button to register new clients.
- Real-time list of clients displaying:
- Client name in primary text.

- Location subtitle: `📍 36.8065, 10.1815` (4-decimal precision) or `📍 Location not captured`.
- Delete icon button for manual removal.
- "Next Client" elevated action button to dequeue the first customer in line.

---

## 4. Testing & Verification Suite

The project includes 17 automated tests across 5 test suites achieving 100% pass rate:

| Test Suite | File | Tests | Coverage Scope | Status |
|---|---|---|---|---|
| **Model Serialization** | `test/client_model_test.dart` | 3 | Serialization, deserialization, location parsing | ✅ Pass |
| **Local SQLite Service** | `test/local_queue_service_test.dart` | 2 | Local insertion, sync status flag updates | ✅ Pass |
| **Queue Geolocation Unit/Widget** | `test/queue_provider_geolocation_test.dart` | 7 | Coordinate persistence, null fallback, UI rendering | ✅ Pass |
| **Remote Sync & Upsert** | `test/queue_provider_test.dart` | 1 | Supabase upsert verification with Mocktail | ✅ Pass |
| **Queue Manager Logic** | `test/waiting_room_manager_test.dart` | 3 | Add client, remove client, nextClient FIFO dequeue | ✅ Pass |
| **UI Widget Interaction** | `test/waiting_room_widget_test.dart` | 4 | Empty state, list rendering, delete and next buttons | ✅ Pass |

### Static Analysis

`dart analyze` output: **No issues found!** (0 errors, 0 warnings, 0 lints).

---

## 5. How to Run the Application

### 5.1 Prerequisites

- Flutter SDK (version 3.44.8 or higher)
- A valid Supabase project with a `clients` table

### 5.2 Environment Setup

10. Copy the example environment file:

```
cp .env.example .env
```

11. Configure your Supabase credentials in `.env`:

```
SUPABASE_URL=https://your-project-id.supabase.co
SUPABASE_ANON_KEY=your-anon-key-here
```

### 5.3 Execution Commands

- **Run on Microsoft Edge**:

```
flutter run -d edge
```

- **Run on Windows Desktop**:

```
flutter run -d windows
```

- **Run on Android Emulator/Device**:

```
flutter run -d android
```

- **Run the Automated Test Suite**:

```
flutter test
```